MINI-PROJET

Un annuaire, quatre algorithmes

Algorithmie comparative en C

Contexte

Vous avez, au fil des exercices précédents, appris à analyser un problème avant de coder, à comparer des algorithmes par leur complexité (Big O), et à manipuler en C les tableaux, les pointeurs, les structures et l'allocation dynamique, et les listes chaînées.

Ce mini-projet réunit tout cela dans un seul programme : un annuaire d'utilisateurs, capable de vérifier si un e-mail existe déjà, implémenté de quatre façons différentes, et comparé à la fois sur le papier et par la mesure.

A faire en binôme et à rendre sur Teams.

Objectif général

Construire, en C, un petit programme qui implémente quatre approches différentes pour rechercher un e-mail dans un annuaire, puis démontrer, par l'analyse et par la mesure, dans quel contexte chacune est la plus adaptée.

Cahier des charges

Fonctionnalités attendues

Le programme final doit permettre, pour chacune des trois approches suivantes :

- ajouter un utilisateur (e-mail + identifiant numérique) à l'annuaire
- vérifier si un e-mail donné existe déjà dans l'annuaire
- mesurer le temps pris par la recherche, pour des annuaires de tailles croissantes

Les trois approches à implémenter

Approche	Structure de données	Rappel du cours
1. Recherche séquentielle	Tableau alloué dynamiquement de struct User	Partie 1 et 2 des exercices précédents
2. Table de hachage	Tableau de listes chaînées (struct Node)	Exercice sur la table de hachage
3. Recherche dichotomique	Tableau trié alloué dynamiquement	Partie 1 des exercices précédents

Contrainte de conception

Les trois approches doivent exposer la même interface, c'est-à-dire les mêmes paramètres en entrée et le même type de retour pour l'insertion et la recherche, même si le nom de la fonction change selon l'approche (seq_, hash_, sorted_).

Par exemple :

```
void seq_insert(const char *email, int id);
bool seq_search(const char *email);

void hash_insert(const char *email, int id);
bool hash_search(const char *email);

void sorted_insert(const char *email, int id);
bool sorted_search(const char *email);
```

Libre à toi d'adapter ces noms et signatures, tant que chaque approche expose clairement une fonction d'ajout et une fonction de recherche.

Étape 1 - Modéliser un utilisateur

Avant de coder les trois approches, définis la structure qui représentera un utilisateur dans les trois cas (elle peut être réutilisée partout, ou déclinée légèrement selon les besoins de chaque approche).

Question 1 - Reprends la grille IOCSF vue en cours. Réponds en une phrase pour chacune des cinq lettres, appliquée à ce projet précis (quelles données, quelle sortie attendue, quelles contraintes, quel volume, quelle fréquence de recherche).

Étape 2 - Implémenter les trois premières approches

Recherche séquentielle

Un tableau de struct User, alloué dynamiquement (realloc ou malloc avec une taille maximale fixée au départ), parcouru linéairement à chaque recherche.

Table de hachage

Un tableau fixe de pointeurs vers des Node (chaînage en cas de collision), comme dans l'exercice dédié lors de la première séance. Choisis une taille de table raisonnable et une fonction de hachage (tu peux réutiliser celle déjà écrite).

Recherche dichotomique

Un tableau de struct User, mais qui doit rester trié par e-mail à tout moment. Réfléchis à la stratégie d'insertion : insères-tu à la bonne place directement, ou ajoutes-tu à la fin puis retries-tu tout le tableau ?

Question 2 - Quelle stratégie d'insertion choisis-tu pour garder le tableau trié, et quel est le coût de cette insertion (pas seulement de la recherche) ?

Points de vigilance

- Chaque approche doit gérer un annuaire vide (aucun utilisateur inséré) sans planter.
- Attention aux fuites mémoire : toute mémoire allouée avec malloc doit être libérée avant la fin du programme.
- Les trois approches doivent être testées séparément avant d'être comparées, pour être sûr qu'elles fonctionnent correctement.

Étape 3 - Protocole de mesure

On veut maintenant mesurer, pour chaque approche, le temps de recherche selon la taille de l'annuaire.

Mesurer le temps en C

La bibliothèque time.h permet de mesurer une durée d'exécution :

```
#include <time.h>

clock_t start = clock();
// ... code à mesurer ...
clock_t end = clock();

double seconds = (double)(end - start) / CLOCKS_PER_SEC;
```

Ce que le script de mesure doit faire

- générer des annuaires de tailles croissantes (par exemple 100, 1 000, 10 000, 100 000, 1 000 000 utilisateurs), avec des e-mails générés automatiquement plutôt que saisis à la main
- pour chaque taille, remplir les trois structures avec les mêmes données
- mesurer, pour chaque approche, le temps d'une recherche dans le pire cas (un e-mail qui n'existe pas dans l'annuaire)

- afficher, ou écrire dans un fichier, un tableau récapitulatif : taille, temps approche 1, temps approche 2, temps approche 3

En clair : *pour générer des e-mails automatiquement, une solution simple est de construire une chaîne à partir d'un compteur, par exemple sprintf(buffer, "user%d@mail.com", i). Pour la recherche dichotomique, pense à générer directement les e-mails dans l'ordre alphabétique, ou à trier le tableau une fois rempli, avant de mesurer la recherche.*

Format de résultat suggéré

Taille de l'annuaire	Séquentielle (s)	Table de hachage (s)	Dichotomique (s)
100	_____	_____	_____
1 000	_____	_____	_____
10 000	_____	_____	_____
100 000	_____	_____	_____
1 000 000	_____	_____	_____

Étape 4 - Analyse comparative

Les mesures seules ne suffisent pas : il faut les interpréter à la lumière de ce que tu as appris sur la complexité.

Question 3 - Avant même de regarder tes mesures, quelle complexité théorique (O(1), O(log n), O(n), etc.) attends-tu pour chacune des trois approches, dans le pire cas ?

Question 4 - Tes mesures confirment-elles cette prédiction ? Si un résultat te surprend, à quoi pourrait-il être dû (facteurs vus dans la dernière partie du cours : constantes cachées, mémoire, environnement) ?

Question 5 - Pour chacune des quatre situations ci-dessous, indique quelle approche tu recommanderais, et justifie en une phrase :

a) Un annuaire de 30 employés, consulté deux fois par jour.

b) Un annuaire de 5 millions d'utilisateurs, interrogé des milliers de fois par seconde.

c) Un annuaire construit une seule fois au démarrage à partir d'un fichier déjà trié, jamais modifié ensuite, mais interrogé très souvent.

d) Un annuaire qui reçoit de nouvelles inscriptions en continu, à un rythme élevé, en plus des recherches.

Étape 5 - Une quatrième approche : liste chaînée

Les trois premières approches reposent sur des tableaux, seuls ou combinés à des listes chaînées pour gérer les collisions de la table de hachage. Ajoute maintenant une quatrième structure, indépendante des trois autres : une liste chaînée simple, où chaque utilisateur est un nœud alloué individuellement et relié au suivant par un pointeur.

Liste chaînée simple

Une struct ListNode contenant un e-mail, un identifiant et un pointeur vers le nœud suivant. L'insertion se fait simplement en tête de liste (coût constant), et la recherche parcourt les nœuds un par un jusqu'à trouver l'e-mail ou atteindre la fin de la liste.

```
typedef struct ListNode {
    char email[100];
    int id;
    struct ListNode *next;
} ListNode;

void list_insert(const char *email, int id);
bool list_search(const char *email);
```

À intégrer au protocole existant

- Comme les trois autres, cette approche doit gérer un annuaire vide sans planter.
- Chaque nœud alloué avec malloc doit être libéré avant la fin du programme.
- Ajoute une colonne à ton protocole de mesure de l'étape 3 pour inclure le temps de recherche de cette quatrième approche, aux mêmes tailles d'annuaire que les trois autres.

Question 6 - Sur le papier, la recherche dans une liste chaînée et la recherche séquentielle sur tableau (approche 1) ont la même complexité au pire cas. Une fois mesurées, leurs temps sont-ils vraiment identiques ? Si un écart apparaît, à quoi peux-tu l'attribuer (indice : bloc contigu en mémoire contre nœuds alloués séparément, localité du cache) ?

En clair : *cette étape ne vise pas à trouver une structure plus rapide, mais à observer un point que la seule notation Big O ne montre pas : deux structures de même complexité théorique peuvent se comporter différemment en pratique selon la façon dont leurs données sont disposées en mémoire.*